

Sequence packing

The cheapest 3× in this book: stop training on padding

Your dashboard says the run is healthy: GPUs busy, MFU respectable, loss falling. Now ask the dashboard a question it doesn't display by default: *what fraction of the tokens you are paying to process are `<pad>`?* For an instruction-tuning set with a typical length profile — median around 900 tokens, mean ~1,300, a long tail to several thousand — padded one-example-per-row to an 8,192-token training length, the answer is **84%**. Five of every six FLOPs, five of every six grad-sync bytes, five of every six dollars: spent teaching the model that padding follows padding. Dynamic per-batch padding helps less than people assume — with shuffled data each batch still contains one long example that sets the row length, and the waste is still ~72%. The fix costs an afternoon of data-pipeline work, requires no new hardware and no quality trade, and is this chapter: pack multiple documents into every row, mask them apart, and collect a 3–6× throughput multiplier that many teams leave on the table for months.

Packing is also this book's cleanest specimen of its favorite species: an efficiency win that lives in the *data pipeline*, not the kernel. Nothing about the model changes. What changes is the answer to "what is a training row?" — and, as always, the details that look bureaucratic (attention masks, position ids, loss normalization) are exactly where correctness lives. We take them in order: how bad the tax is (§16.1), how rows get packed (§16.2), how documents are kept from contaminating each other (§16.3), the kernel that makes the masking free (§16.4), and the SFT-specific accounting that trips people (§16.5).

16.1 The padding tax

The tax exists because document lengths are wildly skewed while the training row is fixed. Web documents, chat transcripts, code files — all roughly log-normal: most short, a heavy tail long. The training row, meanwhile, is one size L , because kernels, memory planning, and parallelism all want static shapes (Chapter 15). Put a skewed distribution into a fixed container one-at-a-time and the container is mostly air:

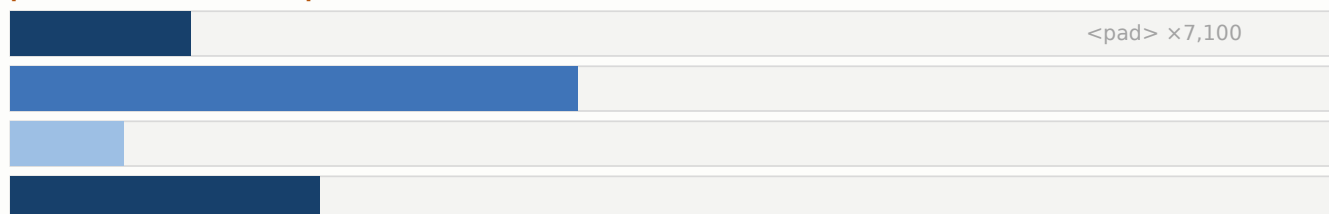
$$\text{waste} = 1 - \frac{\sum_i \ell_i}{N \cdot L}$$

where ℓ_i are the document lengths and N rows are used. The opening's numbers come from exactly this formula on a simulated corpus matching a typical SFT profile (log-normal; median 901, mean 1,314, p95 ~ 4,000, capped at 8,192): one-doc-per-row wastes 84.0%; per-batch dynamic padding with shuffled data still wastes 71.7%, because each batch's

longest member sets its row length. And the two easy dodges both disappoint. *Truncating* to a shorter L throws away the tail — precisely the long-document diet Chapter 21 needs. *Sorting by length* before batching drives waste to $\sim 0.2\%$, but now every batch is length-homogeneous: early batches all tweets, late batches all novels — a curriculum you didn't choose, gradient statistics that drift across the epoch, and a correlation between batch contents and step number that Chapter 18's stability rules dislike. Sorted batching (with bucket-level shuffling) is a fine tool for quick fine-tunes; it is not the principled fix.

THE SAME TRAINING CONTENT, TWO WAYS ($L = 8,192$)

padded — one document per row · 84% air



packed — first-fit-decreasing bins · >99% dense, same kernels



boundaries carry an end-of-document token; §16.3 decides whether attention may cross them

Figure 16.1 — Where the money was going. On the simulated corpus, packing needs 3,209 rows where one-doc-per-row needed 20,000 — $6.2\times$ less compute for identical training content, and $3.5\times$ against dynamic batch-max padding. No model change, no quality trade: the win is purely in what a row contains.

16.2 How rows get packed

Packing is a bin-packing problem — fit documents of lengths ℓ_i into rows of capacity L using as few rows as possible — and the field uses two of its classical answers, chosen by training stage.

Concatenate-and-chunk is pre-training's default, inherited from GPT-3: stream all documents into one endless token sequence with an end-of-document token between them, then slice the stream every L tokens.¹ Waste is exactly zero, the implementation is ten lines, and it has one real cost: documents get *split* at arbitrary chunk boundaries, so a document's tail can land at a row's start with its head in the previous row — context the model never sees. At pre-training scale, with trillions of tokens and documents that mostly fit inside L , the damage is diffuse and long tolerated — but it is not zero: retrieval- and fact-heavy evals improve measurably when truncations are avoided, which is what the "fewer truncations" line of work quantified (best-fit packing cut truncations by orders of magnitude and lifted downstream reading and QA scores at equal compute).²

First-fit-decreasing (FFD) is the fine-tuning and long-context default, where examples are semantic units you must not split (an instruction pair cut in half is garbage supervision). Sort documents longest-first; place each into the first row with room; open a new row when none fits. The classical guarantee says FFD uses at most $\frac{11}{9} \text{ OPT} + \frac{6}{9} \text{ rows}$ ³ — and on real length distributions it does far better than the worst case: on the §16.1 corpus it reaches >99% fill, matching the concatenation baseline while never cutting a document. The whole packer is small enough to read:

```

# first-fit-decreasing packing: no document is ever split
def pack_ffd(examples, L):
    examples = sorted(examples, key=len, reverse=True)      # longest first
    rows, room = [], []                                     # rows of examples;
    space left
    for ex in examples:
        assert len(ex) <= L, "chunk or drop over-length examples upstream"
        for i, r in enumerate(room):
            if len(ex) <= r:                                # first row it fits in
                rows[i].append(ex); room[i] -= len(ex); break
        else:
            rows.append([ex]); room.append(L - len(ex))      # open a new row
    return rows      # feed each row to the collator of §16.4
# >99% fill on typical SFT length profiles; shuffle ROWS afterwards so batch
# composition stays independent of length (the sorted-batching trap of §16.1).

```

Between the two poles sits the streaming compromise most libraries ship as "packing=True": greedily append examples in arrival order, starting a new row when the next one doesn't fit (or, in the laziest variant, chunking anything that overflows). It preserves streaming (no global sort), costs a few points of fill against FFD, and inherits whichever splitting policy you configure — fine for pre-training-style corpora, sloppy for SFT unless splitting is disabled. Know which variant your trainer implements before trusting it: the single most common packing bug report is a team discovering months in that their "packed SFT" was quietly truncating 4% of examples mid-answer.

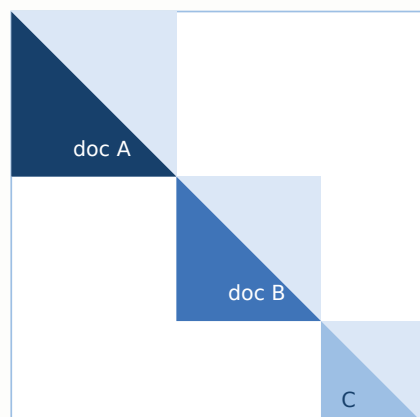
16.3 Keeping documents apart: masks and positions

A packed row holds several documents, and the causal mask alone no longer expresses the truth. Token 1 of document B sits at row position 2,000; plain causal attention lets it read all of document A — a recipe for the model learning that unrelated context precedes everything (and, in instruction tuning, for answers leaking across examples). Two things must reset at every boundary:

The attention mask becomes block-diagonal (within the causal triangle): each document attends only within itself. **Position ids restart from zero** per document, so RoPE distances are within-document distances and the third document in a row doesn't begin at position 5,000. Both are one-line changes in the collator — and both have a famous failure mode, drawn in Figure 16.2, where exactly one of the two is remembered.

TWO RESETS PER BOUNDARY — FORGET EITHER AND THE ROW LIES

attention: block-diagonal $\wedge$ causal



forbidden: B reading A — the blank lower-left stays blank

position ids: restart at every boundary

doc A	doc B	doc C
0 1 2 ... 2047	0 1 2 ... 1499	0 1 ... 899

wrong: 0 1 2 4447 (continuous — RoPE now lies)

Does pre-training need the mask?

History says teams shipped great models without it (GPT-3-style concat lets attention cross EOD). Llama 3 measured it: little effect in standard pre-training, but important for long-context stages — where rows hold few, precious documents.

Default: mask. It costs nothing with \$16.4's kernel.

Figure 16.2 — The two per-boundary resets. Left: the mask is the intersection of causal and block-diagonal — each document is its own triangle. Right: position ids restart per document so rotary distances are within-document truths. Llama 3's ablation found cross-document masking mattered little in ordinary pre-training but was important in long-context stages⁴ — and since the varlen kernel makes masking free, the modern default is simply to always do both.

16.4 The kernel: varlen attention, and the mask that never exists

Here is where packing intersects Chapter 13, and where a naive implementation dies. Materializing the block-diagonal mask as an $L \times L$ tensor and handing it to attention re-creates the exact memory artifact FlashAttention exists to remove (§13.2) — at 8K that is a 64M-entry mask per head, and your "3× speedup" becomes a slowdown plus an OOM. The right interface is the **varlen** (variable-length) entry point: instead of a mask, you pass `cu_seqlens` — the cumulative lengths of the documents in the flattened row, e.g. `[0, 2048, 3548, 4448]` for Figure 16.2's three documents — and the kernel runs each document as its own attention problem, tiled and exact, with nothing materialized. The mask is implicit in the arithmetic. Every serious stack exposes this:

`flash_attn_varlen_func` directly, and in Hugging Face the whole of §16.3 collapses into one collator:⁵

```
# packing, masks, positions, and the varlen kernel — via one collator
from transformers import DataCollatorWithFlattening
collator = DataCollatorWithFlattening() # concatenates the packed row,
# emits cu_seqlens-style boundaries + per-doc position ids, and routes to the
# FlashAttention varlen path. Requires attn_implementation="flash_attention_2".

trainer = Trainer(model, args, train_dataset=pack_ffd(dataset, L=8192),
                  data_collator=collator)

# What NOT to do — the anti-pattern that rebuilds the  $O(L^2)$  problem:
#   attention_mask = build_block_diagonal(L, boundaries) # 64M entries at 8K
#   model(input_ids, attention_mask=attention_mask)      # slow path + OOM
# The boundaries ARE the mask; pass lengths, never a matrix. (Ch. 13, §13.7.)
```

The varlen path carries a bonus that surprises people: with per-document masking, packing doesn't just remove padding FLOPs — it removes *attention* FLOPs. A full row of length L pays $O(L^2)$ pairs; a packed row of documents

$\ell_1, \dots, \ell_k$ pays only $O(\sum_i \ell_i^2)$, because each document is its own attention problem. For six $\sim 1,365$ -token documents filling an 8,192 row, that is $6 \times 1,365^2 \approx 11.2\text{M}$ pairs against $8,192^2 \approx 67.1\text{M}$ — a further **6× cut in attention compute**, on top of the padding win, purely because the kernel now knows where documents end. (The general rule: k equal documents per row cut attention pairs by $k \times$.) This is why "packing with masks" outruns "concatenation without masks" even at identical fill: the unmasked row pays quadratic attention across everything, including across the boundaries where it is learning noise.

16.5 Packing in SFT: the accounting nobody warns you about

Fine-tuning adds two interactions that pre-training doesn't have, both about *weighting* rather than mechanics. First, packing composes with **completion-only loss**: chat training masks the loss on prompt tokens (Chapter 23), and in a packed row the loss mask is the union of each document's completion span — an off-by-one at a boundary silently trains on one prompt token per row, which at scale is a real behavioral smudge. Test the collator on a two-document toy row and *print the labels*; thirty seconds of paranoia, once.

Second, packing quietly changes **what a batch averages over**. With one example per row, per-example influence is roughly equal; token-mean loss over packed rows weights every *token* equally instead — so a 4,000-token essay now carries $\sim 13\times$ the gradient weight of a 300-token exchange. Neither convention is wrong; they are different objectives, and switching to packing switches you between them *silently*. Decide on purpose: token-mean (the packed default) favors long-form quality; example-mean (recoverable by scaling each token's loss by $1/\ell_{\text{doc}}$) preserves the influence of short exchanges — which, for a chat product where most turns are short, is often what you actually want. Put the choice in the config, not in the accident of the data pipeline.

Set expectations for the speedup honestly, because the 6.2× of Figure 16.1 is a *compute* ratio, not a wall-clock promise. Realized gains depend on where your step time actually goes: Hugging Face's measurements with the flattening collator report up to $\sim 2\times$ end-to-end on instruction tuning⁵, with the gap to the theoretical ratio eaten by non-attention overheads, data loading, and optimizer time (Chapter 8's roofline logic — remove one cost and the next one binds). Two knock-on adjustments keep the accounting straight: **tokens per step** jumps by the fill improvement, so revisit batch-size and learning-rate settings in Chapter 17's terms (you may be taking 4× bigger steps than yesterday without having decided to), and **gradient accumulation** counts rows, not examples — a "batch of 64" is now a batch of ~ 250 documents, which changes the noise scale your schedule was tuned against.

16.6 What the big labs do

Table 16.1 — Packing at the frontier.

SYSTEM	STRATEGY	MASKING	NOTE
GPT-3 ¹	concatenate-and-chunk	none across EOD	set the pre-training default for years; zero waste, tolerated contamination

SYSTEM	STRATEGY	MASKING	NOTE
T5 ⁶	multi-example packing	segment-separated	packed with per-segment masking from the start — the SFT pattern, early
Llama 3 ⁴	concat + chunk, doc-masked	block-diagonal	ablated it: minor in standard pre-training, important for long-context stages
Best-fit / AWS study ²	bin packing over chunking	per-document	orders-of-magnitude fewer truncations; measurable eval gains at equal compute
Graphcore recipe ⁷	approximate bin packing	per-document + per-doc positions	the paper that worked out packed BERT end-to-end: ~2× at equal quality
HF ecosystem ⁵	FFD + flattening collator	varlen, implicit	packing + correct masks became a two-line default in 2024
Qwen2.5-1M ⁸	few long docs per row	strict per-document	the Chapter 21 regime: at 256K, what's in the row matters more than fill

16.7 Practical considerations

Measure fill rate and truncation rate as pipeline metrics. Two numbers, logged per dataset build: fraction of row tokens that are real, and fraction of documents split (target: >97% and ~0% for SFT). They catch regressions that nothing else will — a tokenizer change that shifts the length distribution, a new data source with a different tail — and they are the first thing to check when throughput drifts.

Packing changes epoch semantics. "One epoch" of packed rows visits examples in bin-order unless you re-shuffle rows each epoch, and FFD's determinism means identical bins every epoch — re-pack with a shuffled input order (or shuffle within length buckets) per epoch so the model doesn't see the same neighborhoods forever. Cheap, and it removes a subtle memorization channel of co-packed pairs.

Interaction with distributed training is friendly. Static row shapes mean packing composes cleanly with everything in Chapters 14–15 — no dynamic-shape recompiles, no load imbalance across data-parallel ranks (every rank's rows are equally full). This is packing's quiet advantage over dynamic batching schemes that chase the same waste with variable shapes and pay for it in kernel and scheduler churn.

Common pitfalls

Materializing the mask.

The §16.4 anti-pattern: block-diagonal as a tensor rebuilds the $O(L^2)$ memory problem and lands on slow kernels. Pass boundaries, never matrices.

Forgetting position-id resets.

Masks without position resets teach RoPE that document C starts at position 5,000. Silent, and it degrades short-prompt behavior at inference where C starts at 0.

Loss-mask off-by-ones at boundaries.

One prompt token trained per row, times millions of rows. Print the labels on a toy row once per collator change.

Sorted batches without re-shuffling.

The 0.2%-waste seduction of §16.1 — paid for in batch-composition drift. Pack, then shuffle rows.

Silent objective switch.

Token-mean vs example-mean (§16.5) — packing flips it without telling you. Choose in config.

Packing the eval set.

Few-shot evals with packed, cross-visible examples contaminate each other and inflate scores. Evaluation runs unpacked or strictly masked, always.

16.8 Decision framework

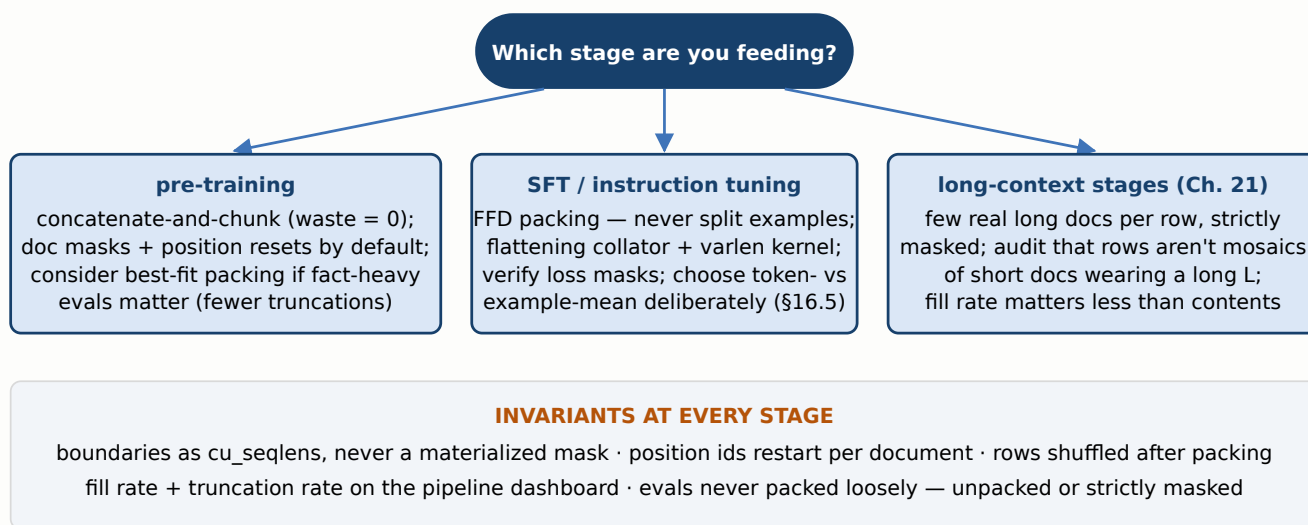


Figure 16.3 — Choosing a packing strategy. Stage picks the packer; the invariants don't move. The one case where packing is genuinely optional: tiny fine-tunes where total compute is minutes — everywhere else the multiplier is too large to decline.

Cost-benefit, bluntly. Packing's ratio is the best in the book: an afternoon of pipeline work and two dashboard metrics buy a 3–6× throughput multiplier on realistic SFT length profiles (and effectively 1.2–2× even in pre-training regimes where documents are longer), with zero model risk when the invariants hold. There is no scale at which it stops paying — **how this scales** is simply that the win tracks your length skew: the shorter your documents relative to L , the bigger

the multiplier, and Chapter 21 pushes L up faster than documents grow. The only real cost is correctness diligence at the three classic seams (mask, positions, loss weights) — which is why this chapter spent its pages exactly there.

16.9 Summary

Padding is a tax on the mismatch between skewed document lengths and fixed training rows, and on realistic instruction-tuning data it consumes most of your compute: 84% waste padding to a fixed 8K, ~72% even with dynamic batch-max padding. Packing eliminates it by making rows dense: concatenate-and-chunk for pre-training (zero waste, tolerated splits — best-fit packing when truncations start costing eval points), first-fit-decreasing for SFT and beyond (>99% fill without ever splitting an example, 11/9-optimal in theory and near-perfect in practice). Correctness lives at the boundaries: attention must be block-diagonal within causal, position ids must restart per document, and both are free when boundaries are passed as `cu_seqlens` to the varlen FlashAttention path instead of ever materializing a mask — the anti-pattern that rebuilds the memory problem Chapter 13 solved. SFT adds two silent decisions: the loss mask's union across packed completions (print the labels once) and the token-mean vs example-mean objective switch that packing performs without asking. Measure fill rate and truncation rate as first-class pipeline metrics, shuffle rows after packing, never pack evals loosely — and collect the cheapest multiplier this book has to offer.

References

1. Brown, T., et al. (2020). Language Models are Few-Shot Learners (GPT-3). *NeurIPS*. arXiv:2005.14165. Concatenate-and-chunk pre-training batches with EOD delimiters.
2. Ding, H., et al. (2024). Fewer Truncations Improve Language Modeling. *ICML*. arXiv:2404.10830. Best-fit bin packing at pre-training scale: massively fewer truncations, improved downstream evals at equal compute.
3. Johnson, D. S. (1973). Near-Optimal Bin Packing Algorithms. MIT PhD thesis. The FFD 11/9-OPT + 6/9 guarantee (constant later tightened by Dósa).
4. Grattafiori, A., et al. (2024). The Llama 3 Herd of Models. arXiv:2407.21783. Cross-document attention masking: limited impact in standard pre-training, important in long-context stages.
5. Hugging Face (2024). Improving Hugging Face Training Efficiency Through Packing with Flash Attention 2 (DataCollatorWithFlattening). HF blog. Packed throughput gains up to ~2× on instruction tuning with correct per-document masking via the varlen path.
6. Raffel, C., et al. (2020). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer (T5). *JMLR* 21. arXiv:1910.10683. Early multi-example packing with segment separation.
7. Krell, M. M., Kosec, M., Perez, S. P., & Fitzgibbon, A. (2021). Efficient Sequence Packing without Cross-contamination. arXiv:2107.02027. The end-to-end packed-training recipe: approximate bin packing, per-document masks and positions, ~2× accelerated BERT at matched quality.
8. Yang, A., et al. (2025). Qwen2.5-1M Technical Report. arXiv:2501.15383. Long-context stages: few genuine long documents per row, strictly masked (see Ch. 21).

Related: *Chapter 13 — Making attention cheap* (the varlen kernel and the never-materialize rule) · *Chapter 21 — Extending context* (packing's long-context regime) · *Chapter 23 — SFT* (completion-only loss) · *Chapter 29 — the case study*. · [Training Efficient LLMs](#)